LAPORAN PRAKTIKUM

TEKNOLOGI IOT II

Oleh :

Nama : LILI CAHYA PUTRI

NPM : 25782071

Kelas : TRI 3C

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

7.5 Praktikum

7.5.1 Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

Langkah Kerja:

1. Buka File Explorer di laptop Anda, buat direktori kerja baru bernama: **iot-server** C:/Users/Users/iot-server.
2. Di dalam folder iot-server, buat struktur sub-direktori berikut secara manual:
   iot-server/
   ├── mosquitto/
   │ └── config/
3. Buka teks editor (VS Code atau Notepad), buat berkas baru bernama **mosquitto.conf**, simpan di dalam folder iot-server/mosquitto/config/, lalu masukkan konfigurasi parameter berikut:

```
# ================================================================================
# KONFIGURASI DASAR ECLIPSE MOSQUITTO BROKER
# ================================================================================

# Listener Saluran MQTT Standar (Untuk NodeMCU ESP8266 & Klien TCP)
listener 1883
protocol mqtt

# Listener Saluran WebSockets (Untuk Aplikasi Web Browser & Dashboard Visual)
listener 9001
protocol websockets

# Perizinan Akses Awal (Tanpa Password)
allow_anonymous true

# Konfigurasi Persistensi Penyimpanan Pesan
persistence true
persistence_location /mosquitto/data/

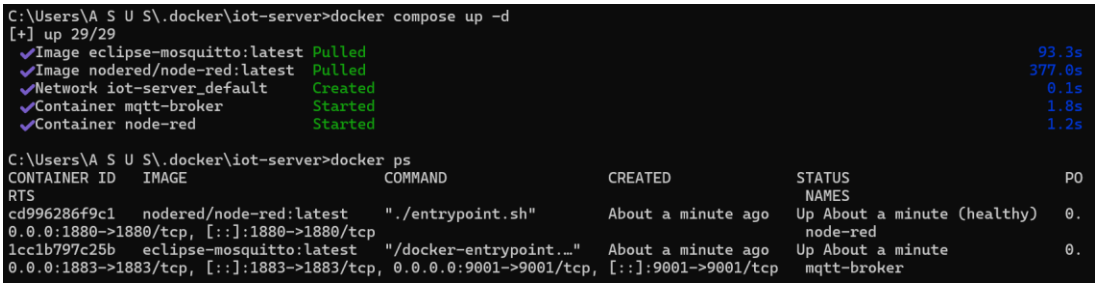
# Konfigurasi Berkas Log
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

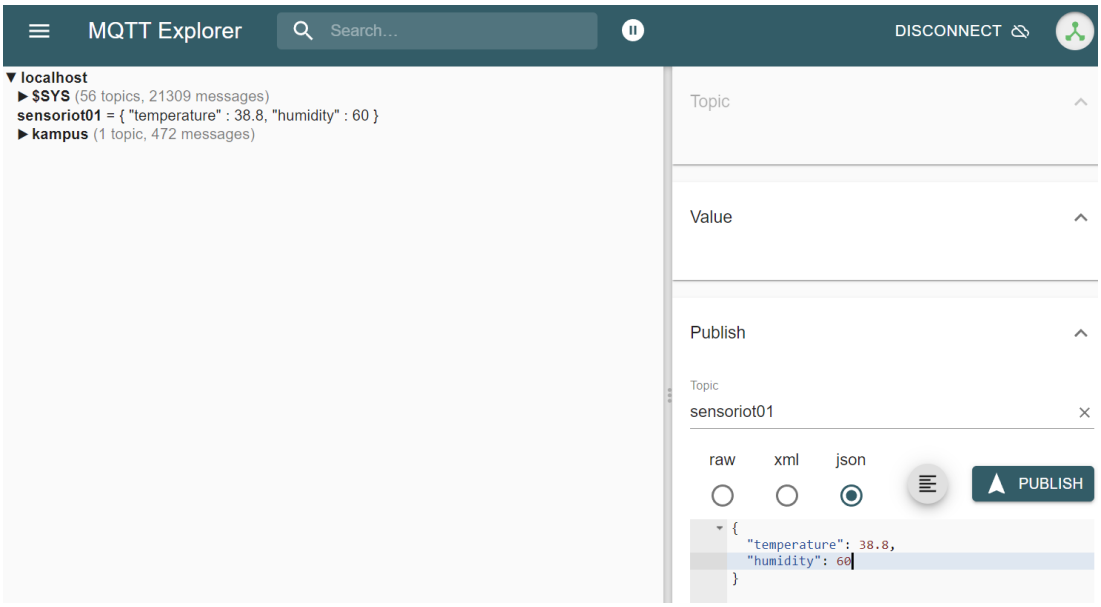
4. Kembali ke folder induk iot-server, buat berkas baru bernama **docker-compose.yml**, lalu salin konfigurasi layanan terpadu berikut:

```
services:
  # ============================================================
  # 1. Layanan Broker MQTT: Eclipse Mosquitto
  # ============================================================
  mqtt:
    image: eclipse-mosquitto:latest
    container_name: mqtt-broker
    restart: unless-stopped
    ports:
      - "1883:1883"  # Port MQTT murni untuk mikrokontroler
      - "9001:9001"  # Port WebSockets untuk dashboard web
    volumes:
      - ./mosquitto/config:/mosquitto/config
      - ./mosquitto/data:/mosquitto/data
      - ./mosquitto/log:/mosquitto/log

  # ============================================================
  # 2. Layanan Visual Rule Engine: Node-RED
  # ============================================================
  nodered:
    image: nodered/node-red:latest
    container_name: node-red
    restart: unless-stopped
    ports:
      - "1880:1880"  # Port antarmuka web builder Node-RED
    environment:
      - TZ=Asia/Jakarta
    volumes:
      - ./nodered:/data
    depends_on:
      - mqtt
```

5. Jalankan **Terminal** (Linux/macOS) atau **PowerShell / Command Prompt** (Windows), arahkan direktori aktif ke dalam folder iot-server:
6. Eksekusi perintah berikut untuk mengunduh (*pull*) *image* dan menjalankan kontainer di latar belakang (*detached mode*): docker compose up -d
7. Tunggu hingga terminal menampilkan status [+] Running 2/2 dengan keterangan hijau Started.





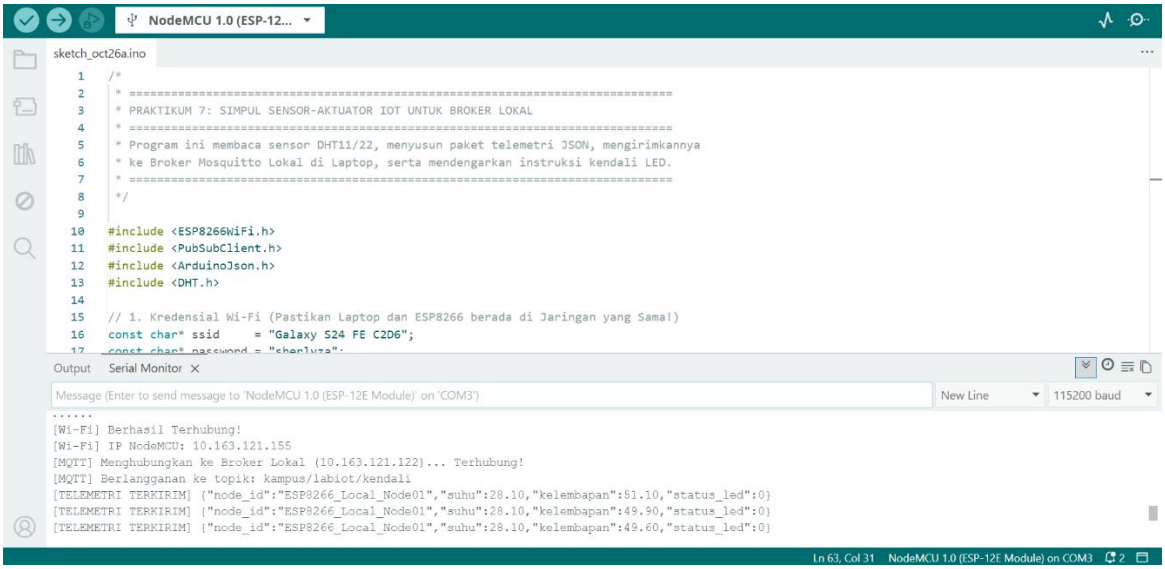
8. Verifikasi bahwa kedua kontainer telah aktif dan berjalan normal dengan mengecek daftar proses kontainer: docker ps
9. Buka peramban web pada laptop Anda dan ketikkan alamat URL: **http://localhost:1880**. Jika antarmuka visual kanvas Node-RED berhasil tampil di layar, maka fondasi server IoT lokal Anda telah siap beroperasi secara sempurna!

7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

Langkah Pemrograman:

1. Cari tahu Alamat IPv4 Lokal laptop Anda yang terhubung ke jaringan Wi-Fi:
    - Windows: Buka CMD / PowerShell, ketik ipconfig. Temukan bagian *Wireless LAN adapter Wi-Fi*, lalu catat baris IPv4 Address (misalnya: 192.168.1.15).

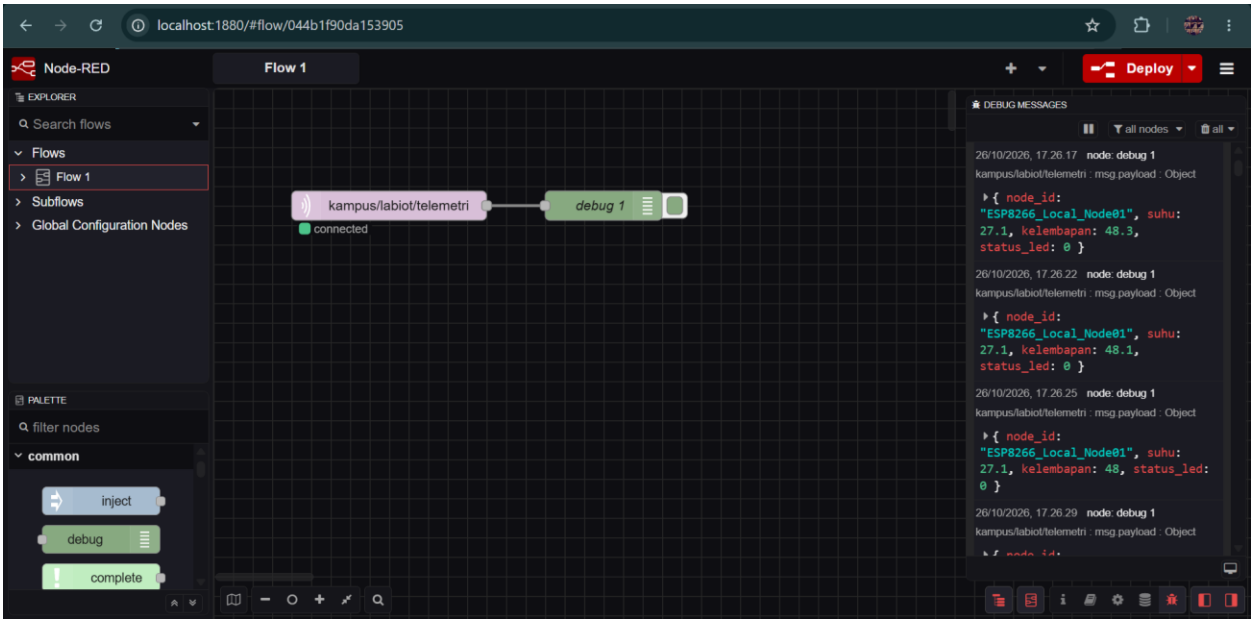
2. Buka **Arduino IDE**, buat berkas sketsa baru, lalu salin kode program terpadu berikut:
3. Unggah (*Upload*) kode program ke board NodeMCU ESP8266.
4. Buka **Serial Monitor** pada kecepatan 115200 baud. Pastikan mikrokontroler berhasil mendapatkan alamat IP dan memunculkan tulisan [MQTT] Menghubungkan ke Broker Lokal... Terhubung! serta mulai mengirimkan payload telemetri JSON secara stabil.



7.5.3 Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED

Langkah Kerja:

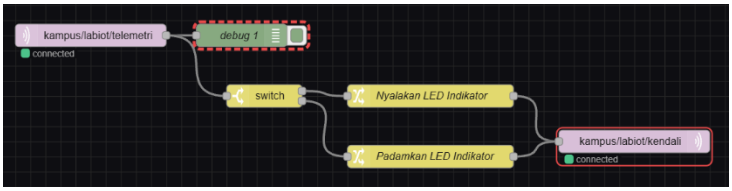
1. peramban web dan akses antarmuka Node-RED di **http://localhost:1880**.
2. Menambahkan Konfigurasi Server MQTT Broker dan lihat outputnya



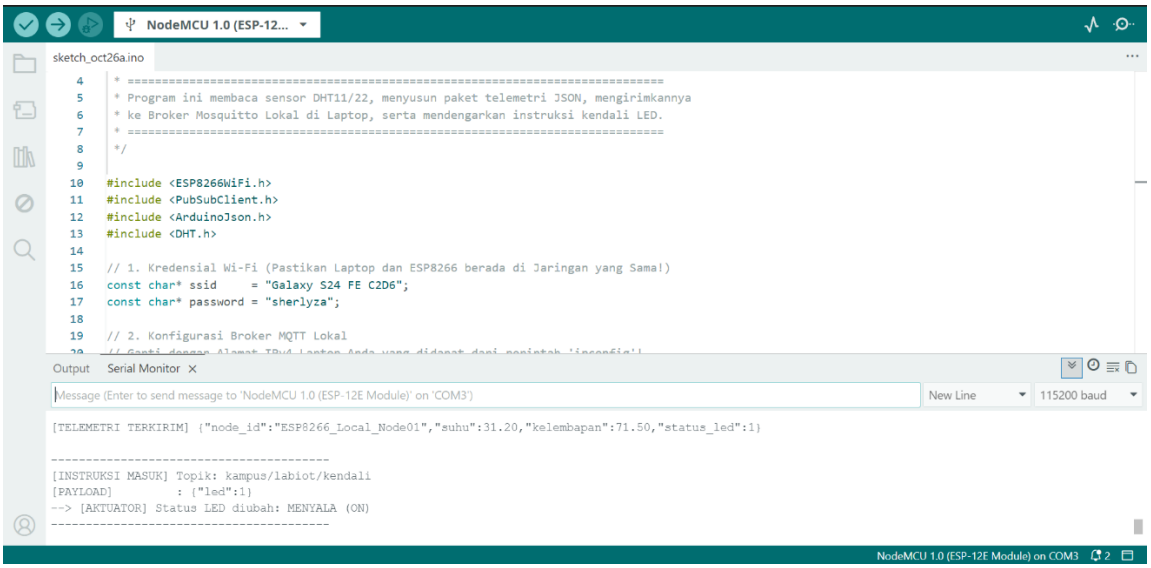
7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup

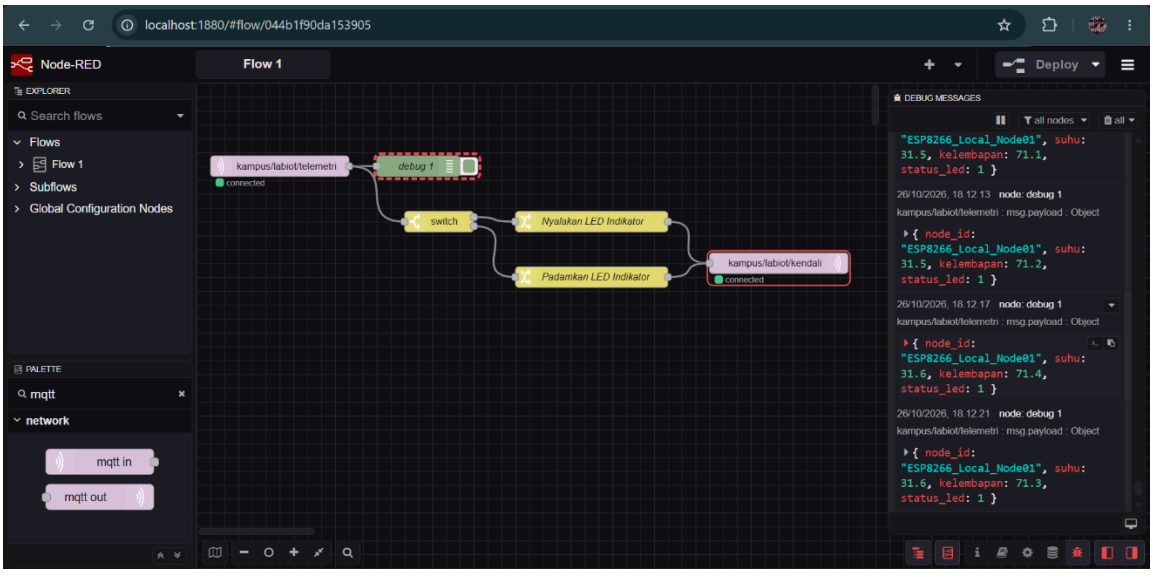
Langkah Kerja:

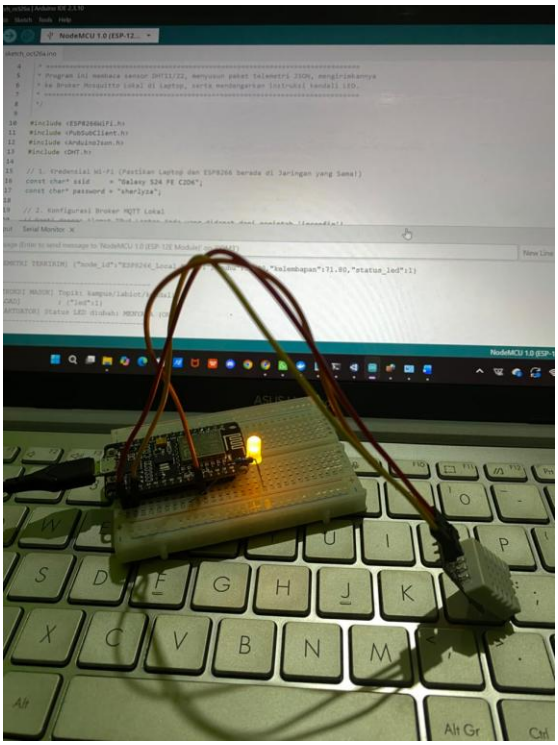
1. Membuat Node Percabangan Logika (switch):
2. Membuat Node Transformasi Muatan Panas (change):
3. Membuat Node Transformasi Muatan Normal (change):
4. Menerbitkan Instruksi Balik ke Broker (mqtt out):
5. Menerapkan Perubahan (*Deploy*):



6. Pengujian Fisik Sistem Otomasi Tertutup:







7.6 Latihan

- **Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery):**
  Pada Praktikum 3, di dalam konfigurasi node mqtt in pada Node-RED, kita mengisi alamat server broker dengan nama teks **mqtt**, bukan alamat IP lokal laptop (192.168.1.15) maupun localhost.

  i. Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (*Connection Refused*)?

  Di dalam arsitektur Docker, setiap kontainer berjalan di dalam *network namespace* yang terisolasi. Kata kunci localhost (atau 127.0.0.1) di dalam kontainer Node-RED merujuk pada *loopback interface* dari kontainer Node-RED itu sendiri, bukan merujuk ke laptop (host) ataupun kontainer lainnya. Oleh karena itu, ketika Node-RED mencoba mencari broker MQTT pada localhost:1883, ia mencari di dalam sistem kontainernya sendiri di mana broker Mosquitto tidak diinstal atau tidak berjalan, sehingga menghasilkan *Connection Refused*.

  ii. Jelaskan bagaimana mekanisme *Docker Embedded DNS Server* dan *Bridge Network* bekerja sehingga kontainer node-red dapat menemukan kontainer mqtt-broker cukup dengan memanggil nama layanannya!

Di dalam arsitektur Docker, setiap kontainer berjalan di dalam *network namespace* yang terisolasi. Kata kunci localhost (atau 127.0.0.1) di dalam kontainer Node-RED merujuk pada *loopback interface* dari kontainer Node-RED itu sendiri, bukan merujuk ke laptop (host) ataupun kontainer lainnya. Oleh karena itu, ketika Node-RED mencoba mencari broker MQTT pada localhost:1883, ia mencari di dalam sistem kontainernya sendiri di mana broker Mosquitto tidak diinstal atau tidak berjalan, sehingga menghasilkan *Connection Refused*.

- **Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall):**
Seorang mahasiswa mencoba menghubungkan NodeMCU ESP8266 ke broker lokal di laptopnya. ESP8266 berhasil terhubung ke Wi-Fi dan mendapatkan alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15.

  i. Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?

  Kedua perangkat berada pada **subnet/segmen jaringan yang berbeda**. NodeMCU berada di subnet 192.168.43.x (biasanya dari hotspot HP), sedangkan laptop berada di subnet 192.168.1.x (dari router Wi-Fi yang berbeda). Perangkat yang berada pada jaringan fisik atau subnet yang berbeda tidak dapat saling berkomunikasi secara langsung tanpa adanya perangkat *router* yang melakukan routing atau *port forwarding*, sehingga paket data MQTT gagal dikirimkan.

  ii. Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain *broadcast / subnet* yang sama?

  - **Menyamakan Sumber Jaringan:** Ubah koneksi Wi-Fi laptop agar terhubung ke jaringan hotspot yang sama dengan NodeMCU (`192.168.43.x`), atau sebaliknya, hubungkan NodeMCU ke jaringan Wi-Fi tempat laptop terhubung (`192.168.1.x`).
  - **Memperbarui Konfigurasi IP:** Setelah kedua perangkat terhubung ke SSID/jaringan yang sama, cek kembali IPv4 laptop menggunakan perintah `ipconfig` (Windows) atau `ifconfig` (Linux/macOS), lalu perbarui nilai variabel `mqttServer` pada program Arduino IDE NodeMCU agar sesuai dengan IP terbaru laptop tersebut.

- **Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):**
Pada sistem kendali dua posisi (*Bang-Bang*), jika suhu berfluktuasi cepat di sekitar ambang batas 31.0°C(misal: 31.05°C → 30.95°C → 31.02°C), lampu LED indikator akan berkedip-kedip cepat (*flickering / hunting*) yang mengganggu stabilitas visual sistem.

Rancang modifikasi alur Node-RED menggunakan **function node** berbahasa JavaScript yang menerapkan logika **Histeresis**:

- LED baru menyala jika suhu $> 31.5°C$(msg.payload = {"led": 1}).
- LED baru padam jika suhu mendingin hingga $< 30.0°C$(msg.payload = {"led": 0}).
  Tuliskan kode JavaScript yang Anda masukkan ke dalam *Function Node* tersebut!

Kode JavaScript yang dimasukkan ke dalam **Function Node** untuk menerapkan logika Histeresis (mencegah *flickering* atau *hunting* pada kendali LED):

```
// Variabel global untuk menyimpan status state LED terakhir (1 = Nyala, 0 = Padam)
// Disimpan menggunakan context.get agar nilainya persis dipertahankan antar-eksekusi message
let ledState = context.get("ledState") || 0;

// Mengambil nilai suhu dari msg.payload (disesuaikan dengan struktur JSON telemetri)
let suhu = parseFloat(msg.payload.suhu);

if (!isNaN(suhu)) {
    // Logika Histeresis
    if (suhu > 31.5) {
        ledState = 1; // Nyalakan LED jika suhu melebihi batas atas
    } else if (suhu < 30.0) {
        ledState = 0; // Padamkan LED jika suhu turun hingga di bawah batas bawah
    }
    // Jika suhu berada di rentang 30.0 - 31.5 (zona aman/deadband), status ledState dipertahankan

    // Simpan kembali state ke konteks
    context.set("ledState", ledState);

    // Keluarkan payload kendali format JSON untuk node MQTT out
    msg.payload = { "led": ledState };
    return msg;
}

return null; // Abaikan jika data suhu tidak valid
```

- **Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data):**
  Pada berkas docker-compose.yml, terdapat pemetaan direktori - ./nodered:/data dan - ./mosquitto/data:/mosquitto/data.

  i. Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah docker compose down, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari docker-compose.yml?

  Seluruh data konfigurasi, *flow* kanvas visual, kredensial, dan riwayat yang tersimpan di dalam direktori /data kontainer akan **hilang permanen**. Ketika kontainer dinyalakan kembali, Node-RED akan kembali ke kondisi awal (bersih seperti baru diinstal) karena tidak ada data yang disimpan secara persisten di luar kontainer (*ephemeral storage*).

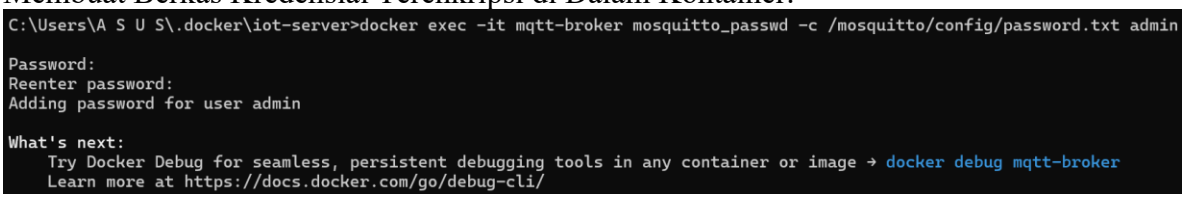
  ii. Jelaskan perbedaan mendasar antara *Bind Mount* dan *Named Volume* pada Docker!

  - **Bind Mount:** Mengaitkan (*mount*) direktori atau file tertentu dari **sistem berkas host (laptop/komputer lokal)** ke dalam kontainer secara langsung. Jalur absolut pada host ditentukan secara manual oleh pengguna (contoh: `./nodered:/data`).
  - **Named Volume:** Pengelolaan penyimpanan data sepenuhnya dikendalikan dan diatur oleh Docker di dalam area penyimpanan khusus di dalam sistem host (biasanya di dalam direktori internal Docker, misal `/var/lib/docker/volumes/...`). *Named Volume* lebih direkomendasikan untuk portabilitas karena tidak bergantung pada struktur direktori spesifik di komputer lokal.

7.7 Tugas: Pengamanan Server IoT Industri (MQTT Authentication & Access Control)

Instruksi Pengerjaan Langkah demi Langkah:

1. Membuat Berkas Kredensial Terenkripsi di Dalam Kontainer:



2. Memperbarui Konfigurasi Kebijakan Mosquitto:

```
# ================================================================================
# KONFIGURASI DASAR ECLIPSE MOSQUITTO BROKER
# ================================================================================

# Listener Saluran MQTT Standar (Untuk NodeMCU ESP8266 & Klien TCP)
listener 1883
protocol mqtt

# Listener Saluran WebSockets (Untuk Aplikasi Web Browser & Dashboard Visual)
listener 9001
protocol websockets

# Perizinan Akses Awal (Tanpa Password)
allow_anonymous false
#password_file /mosquitto/config/password.txt

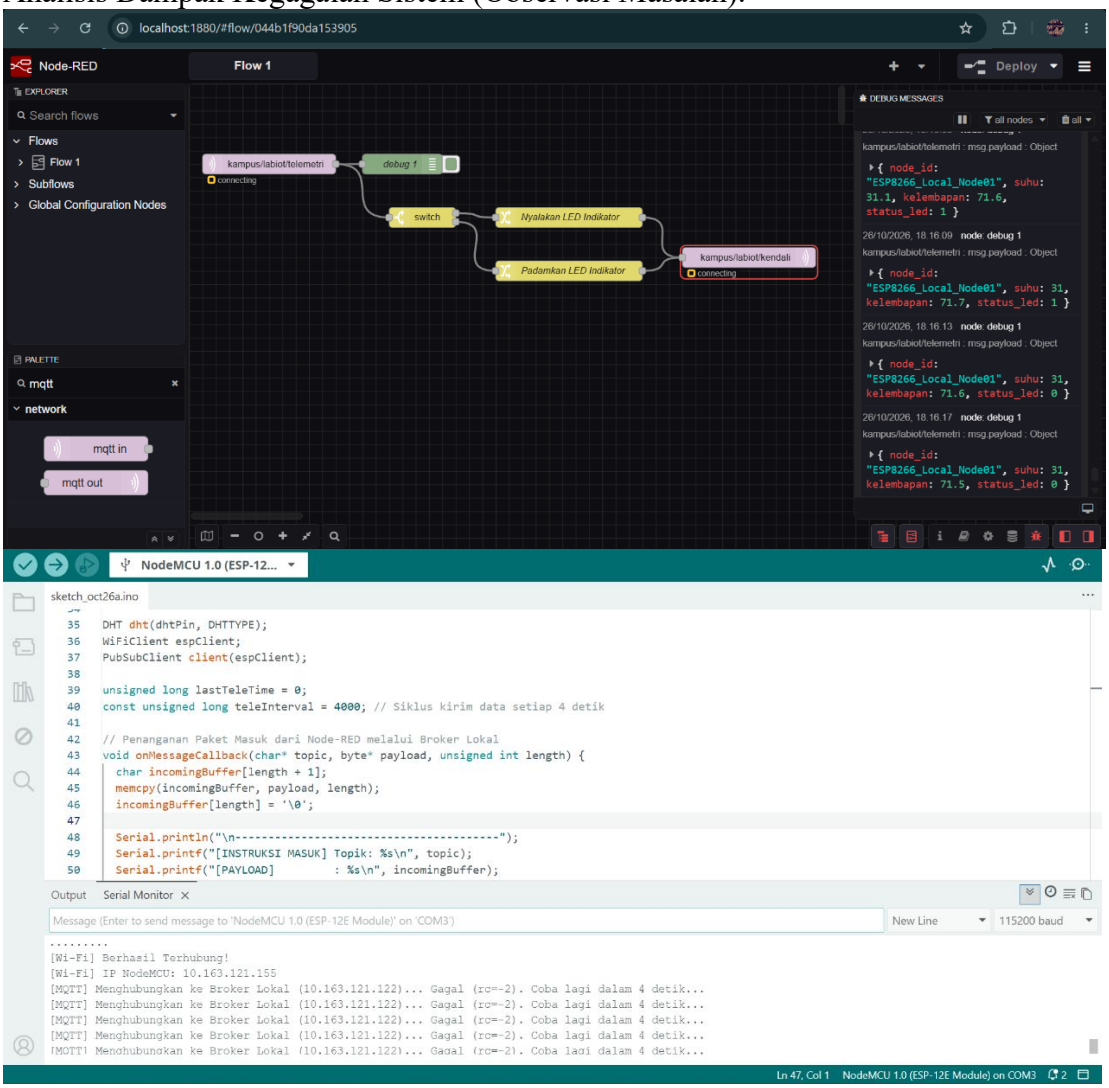
# Konfigurasi Persistensi Penyimpanan Pesan
persistence true
persistence_location /mosquitto/data/

# Konfigurasi Berkas Log
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

3. Memuat Ulang Layanan Broker:

```
C:\Users\A S U S\.docker\iot-server>docker restart mqtt-broker
mqtt-broker
```

4. Analisis Dampak Kegagalan Sistem (Observasi Masalah):



5. Rekayasa Perbaikan Sistem (Penyelesaian Tugas):

